JSSMAHAVIDYAPEETHA

JSS ACADEMY OF TECHNICAL EDUCATION

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

JSSATE Campus, Dr. Vishnuvardhana Main Road, Bengaluru – 560060

KRISHI: AI-Powered Agriculture Assistant

FINAL YEAR PROJECT WORK(BIS786)

SYNOPSIS

SUBMITTED BY

Ananya Singh– 1JS23IS012
Harsh Kumar Anand – 1JS23IS050
Dhriti RR– 1JS23IS040
Harshdeep Jadhav – 1JS23IS054

2026-27

Under the Guidance of

Dr Sahana V

(Associate Professor)

Project Coordinators

Dr. Rekha P M, Professor
Dr. Sreenatha M, Associate Professor
Mrs. Deepa Konnur, Assistant Professor

HOD

Dr. Nagamani N Purohit, Associate Professor

Contents

Introduction

1.1 Background of the Topic

Agriculture is the backbone of the Indian economy, supporting over 120 million farming households and employing more than 600 million people. Despite its importance, Indian farmers face critical challenges including delayed crop disease diagnosis, language barriers, limited internet connectivity in rural areas, and a fragmented ecosystem of digital tools. Crop diseases alone account for 20–40% annual yield losses, and the lack of timely, accessible expert guidance significantly exacerbates the problem.

Traditional agricultural advisory systems rely on government extension workers or physical expert visits — approaches that are slow, expensive, and unable to scale to the vast number of smallholder farmers. Most digital tools currently available are English-only, cloud-dependent, and address only isolated aspects of farming, failing to provide a comprehensive solution.

KRISHI (Sanskrit: कृषि, meaning Agriculture) is an AI-powered, mobile-friendly web application designed to bridge this gap. Built using the MERN stack (MongoDB, Express.js, React 18, Node.js), Krishi integrates AI crop disease detection, multilingual support (English, Hindi, Kannada), a peer-to-peer marketplace, expert consultation, community forum, and a 24/7 AI chatbot — all in a single unified platform.

1.2 Relevance of the Problem

In modern rural India, the need for intelligent, accessible, and offline-capable agricultural systems is increasingly critical. The absence of a unified platform creates a significant gap in farming support infrastructure:

- Crop disease outbreaks go undetected due to the absence of timely diagnosis tools
- Millions of Hindi, Kannada, Tamil, and other regional language speakers are excluded by English-only tools
- Rural farming areas with limited internet connectivity cannot rely on cloud-only solutions
- Farmers must juggle multiple fragmented applications for different needs
- High post-harvest losses and yield reductions result from lack of actionable guidance

1.3 Importance of the Study

This study is important because it proposes an intelligent, offline-first system that bridges the gap between farmers and modern agricultural knowledge. By leveraging computer vision, deep learning, NLP, and a Progressive Web App (PWA) architecture, Krishi can:

- Automatically detect crop diseases from leaf images with approximately 92% accuracy
- Provide chemical and organic treatment recommendations in real-time
- Support English, Hindi, and Kannada across all modules via the Web Speech API
- Operate fully offline using local knowledge bases (plantDiseases.js, agriKnowledge.js)
- Connect farmers with certified agricultural experts through an integrated consultation module
- Enable direct peer-to-peer commerce through an agricultural marketplace
- Foster community knowledge sharing through a moderated forum

1.4 Current Scenario in Industry and Academia

Modern agricultural technology systems primarily focus on:

- Plant disease detection using CNN models (MobileNet, ResNet, VGG16) trained on the PlantVillage dataset
- IoT-based crop monitoring with sensor networks for real-time field data
- Single-purpose mobile apps for disease classification, lacking integrated advisory features
- Cloud-based AI inference requiring constant internet connectivity

However, most existing solutions lack:

- Offline-first capability suitable for low-connectivity rural environments
- Multilingual support for regional Indian languages
- Integration of diagnosis, expert consultation, marketplace, and community into one platform
- Voice-based accessibility for non-literate or low-literacy farmers
- Scalable peer-to-peer marketplaces connecting farmers directly to buyers

There is growing academic and industry interest in developing edge AI systems for agriculture that can provide intelligent insights while remaining accessible to farmers in the most remote areas.

Literature Review

2.1 AI-Based Crop Disease Detection

2.1.1 Deep CNN for Plant Disease Detection

Authors: Mohanty et al. (2023)

Methodology: Convolutional Neural Networks trained on the PlantVillage dataset for automated multi-class crop disease classification from leaf images.

Key Concepts: Multi-class image classification, transfer learning, data augmentation, confidence scoring.

Limitation: Requires large labeled datasets; real-world performance degrades in unstructured rural environments.

Research Gap: Does not address offline deployment, multilingual output, or integration with advisory systems.

2.1.2 Transfer Learning for Leaf Disease Recognition

Authors: Too et al. (2022)

Methodology: MobileNet-based transfer learning for lightweight disease classification suitable for mobile deployment.

Key Concepts: Feature extraction, fine-tuning, lightweight architecture, mobile inference.

Limitation: Limited generalization to unseen disease variants and crop species outside training distribution.

Research Gap: No offline functionality; does not address regional language accessibility or farmer-facing UX.

2.1.3 CNN-Based Tomato Disease Detection

Authors: Durmuş et al. (2022)

Methodology: Deep CNN trained for tomato disease detection with high category-specific accuracy.

Key Concepts: Crop-specific model training, bounding box localization, confidence thresholding.

Limitation: Restricted to tomato crops; dataset limitations affect generalization.

Research Gap: Single-crop, single-language, cloud-dependent system with no advisory or marketplace integration.

2.1.4 Image Classification using EfficientNet

Authors: Tan et al. (2024)

Methodology: EfficientNet compound scaling approach for scalable and accurate plant disease classification.

Key Concepts: Compound model scaling, accuracy-efficiency trade-off, mobile deployment.

Limitation: Requires significant compute for training and hyperparameter tuning.

Research Gap: Does not address multilingual output, offline-first architecture, or integrated farmer advisory.

2.2 Offline and Edge AI for Agriculture

2.2.1 Edge AI for Offline Plant Disease Detection

Authors: Sharma et al. (2024)

Methodology: Edge computing deployment of lightweight AI models for real-time inference without internet connectivity.

Key Concepts: On-device inference, model compression, TensorFlow.js, low latency processing.

Limitation: Limited device compatibility; storage constraints restrict model complexity.

Research Gap: No unified platform; lacks marketplace, consultation, or multilingual support.

2.2.2 Progressive Web Apps for Offline Web Systems

Authors: (Technical Paper, 2020)

Methodology: Service worker-based PWA architecture enabling offline-first web application delivery.

Key Concepts: Cache-first strategy, background sync, installable web apps, Lighthouse performance scoring.

Limitation: Complex service worker lifecycle; storage quotas limit offline data volume.

Research Gap: General PWA framework not adapted to agricultural domain or multilingual voice output.

2.3 Multilingual and Chatbot Systems

2.3.1 AI Chatbot for Farmer Assistance

Authors: Kumar et al. (2023)

Methodology: NLP-based conversational agent providing agriculture advisory through natural language interaction.

Key Concepts: Intent classification, entity extraction, contextual response generation, knowledge base querying.

Limitation: Limited conversational context; primarily English-only with no regional language support.

Research Gap: Standalone chatbot not integrated with disease detection, marketplace, or expert consultation.

2.3.2 Multi-Language AI Systems for Agriculture

Authors: Reddy et al. (2024)

Methodology: Multilingual NLP models supporting regional Indian language input and output for agricultural advisory.

Key Concepts: Language detection, machine translation, regional script rendering, voice synthesis.

Limitation: Translation errors for domain-specific agricultural terminology; accuracy varies across languages.

Research Gap: Language module not integrated with offline plant disease detection or P2P marketplace.

Problem Statement

3.1 Statement of the Problem

Crop disease diagnosis and agricultural advisory for Indian smallholder farmers is a complex and inefficient process. Traditional systems and digital tools operate in isolation with the following limitations:

- Delayed Diagnosis: Farmers lack timely access to expert crop disease diagnosis, causing massive yield losses before treatment begins
- Language Barriers: Most agricultural tools are English-only, excluding millions of Hindi, Kannada, Tamil, and regional language speakers
- Connectivity Issues: Rural farming areas have limited internet, making cloud-only solutions ineffective or completely inaccessible
- Fragmented Ecosystem: No single platform integrates disease diagnosis, expert consultation, P2P marketplace, and community support
- Low Accessibility: Existing solutions require technical literacy and do not support voice-based interaction for low-literacy users
- No Offline Capability: Current AI-based systems fail entirely when internet connectivity is unavailable

3.2 Gaps in Existing Systems

Current agricultural technology systems have several critical limitations that Krishi aims to address:

Lack of Offline AI: Systems do not function without internet; no on-device inference capability for rural deployment

No Unified Platform: Farmers need separate apps for diagnosis, marketplace, consultation, and community interaction

English-Only Interfaces: Regional language support is absent, creating a digital divide for non-English speakers

No Voice Accessibility: Absence of voice synthesis for farmers with low digital literacy

Limited Treatment Guidance: Systems provide only disease labels without actionable organic or chemical treatment recommendations

No Community Layer: Existing tools are individual-use only, lacking peer knowledge sharing or community forums

No Direct Commerce: Farmers cannot sell produce or buy agricultural inputs within the same platform

3.3 Problem Definition

There is a critical need for an intelligent, offline-capable, multilingual agricultural assistant that can:

- Automatically detect crop diseases from leaf images with high accuracy using on-device AI
- Provide actionable chemical and organic treatment recommendations with confidence scoring

- Support multiple Indian languages — English, Hindi, and Kannada — across all modules
- Operate in low or zero internet connectivity environments through an offline-first PWA architecture
- Connect farmers with certified agricultural experts for paid consultation
- Enable a peer-to-peer marketplace for agricultural produce and inputs
- Provide a community forum for knowledge sharing among farmers
- Offer a 24/7 AI chatbot for instant agricultural query resolution

The absence of such a comprehensive, accessible, and offline-capable system represents a significant gap in current Indian agricultural technology infrastructure.

Objective of the Study

The primary objective of this project is to develop KRISHI, an intelligent, multilingual, offline-first web application that empowers Indian farmers with AI-powered crop disease diagnosis, agricultural advisory, peer-to-peer commerce, expert consultation, and community support in a single unified platform.

4.1 Specific Objectives

Real-Time AI Crop Disease Detection

- Implement a computer vision model to identify crop diseases from leaf images
- Classify diseases across 15+ categories with approximately 92% accuracy on test data
- Generate disease predictions with confidence scores for user transparency

AI-Powered Treatment Recommendations

- Provide actionable chemical treatment recommendations post-diagnosis
- Suggest organic and natural remedies as alternative treatment options
- Include prevention tips and risk factor explanations for farmer education

Multilingual Support

- Implement full multilingual UI supporting English, Hindi, and Kannada
- Integrate Web Speech API for voice synthesis output in all three languages
- Enable language switching across all modules without data loss

Offline-First Architecture

- Deploy as a Progressive Web App (PWA) using service workers and cache-first strategy
- Maintain a local knowledge base (plantDiseases.js, agriKnowledge.js) for offline queries
- Sync data automatically when connectivity is restored
- Target a Lighthouse performance score of 98/100

Expert Consultation Module

- Build a directory of certified agricultural experts with profiles and specializations
- Enable farmers to book and conduct expert consultations through the platform
- Maintain consultation history and expert ratings for quality assurance

Peer-to-Peer Agricultural Marketplace

- Develop a P2P marketplace for buying and selling agricultural produce and inputs
- Implement product listing, search, and inquiry management features
- Integrate with MongoDB Atlas products collection via REST API

Community Forum

- Build a community forum for farmer knowledge sharing and peer support
- Enable threaded posts, replies, and moderation capabilities
- Integrate with MongoDB Atlas posts collection via /api/community endpoint

24/7 AI Chatbot

- Deploy an AI-powered chatbot for instant agricultural query resolution
- Support contextual, agriculture-specific responses using agriKnowledge.js knowledge base
- Enable multilingual chatbot interaction with voice output support

System Performance and Scalability

- Achieve end-to-end scan-to-result flow in under 3 seconds
- Support multiple concurrent users through efficient API and frontend design
- Deploy on Vercel for scalable cloud hosting with lightweight frontend optimization

Scope of the Project

5.1 System Scope

The proposed Krishi system is designed to provide a comprehensive, offline-capable agricultural assistance platform for Indian smallholder farmers. The system encompasses:

- Mobile-friendly web application accessible via browser on smartphone and desktop
- AI-based crop disease diagnosis from camera-captured or uploaded leaf images
- Offline-first functionality with local knowledge base and PWA caching
- Multilingual UI and voice synthesis in English, Hindi, and Kannada
- Peer-to-peer agricultural marketplace for produce and input trading
- Certified expert consultation booking and management
- Community forum for farmer knowledge sharing
- 24/7 AI chatbot for instant agricultural advisory
- REST API backend with /api/chat, /api/community, /api/market, /api/consult endpoints
- MongoDB Atlas cloud database with experts, products, and posts collections

5.2 Target Domain and Applications

The system is designed for deployment in the following contexts:

- Smallholder and marginal farmers in rural India
- Agricultural extension workers and field advisors
- Certified agricultural experts offering remote consultation
- Agricultural input suppliers and produce buyers using the marketplace
- Farming communities and cooperative societies
- State agriculture departments and farmer producer organizations (FPOs)
- Agricultural research institutions for farmer-facing digital tool deployment

5.3 Technology Stack

5.3.1 Frontend

- Framework: React 18 with functional components and hooks
- Styling: Glassmorphism UI with Tailwind CSS and custom CSS
- Voice: Web Speech API for multilingual voice synthesis
- Camera: Browser-based camera access for leaf scanning
- PWA: Service workers with cache-first strategy for offline support

5.3.2 Backend and Infrastructure

- Runtime: Node.js with Express.js REST API framework
- Database: MongoDB Atlas with experts, products, and posts collections
- Hosting: Vercel (frontend and serverless backend deployment)
- Image handling: Multer for upload management, Sharp for preprocessing

5.3.3 AI and Machine Learning

- Disease Detection: TensorFlow.js for in-browser model inference (model.json + .bin files)
- Offline Knowledge Base: plantDiseases.js (15+ disease categories), agriKnowledge.js
- Chatbot: AI-powered NLP with agriKnowledge.js fallback for offline mode
- Image preprocessing: Resize to 224×224, normalize pixels before model inference

5.3.4 Additional Libraries

- Web Speech API (voice synthesis for multilingual output)
- Mongoose (MongoDB object modeling)
- Axios (HTTP client for API calls)
- React Router (client-side navigation)

5.4 Limitations of the Project

- Real CNN model training is out of scope; implementation uses TensorFlow.js with pre-trained weights
- Authentication and JWT-based user login are listed as future enhancements
- PWA service workers are partially implemented; full production PWA is a future milestone
- WhatsApp API integration for notifications is planned for a future release
- Disease detection accuracy depends on image quality and lighting conditions in the field
- Chatbot context window is limited; multi-turn long conversations may lose earlier context
- Marketplace payment gateway integration is not included in current scope
- The system requires initial internet access to download TensorFlow.js model files on first use

5.5 Future Enhancements

- Full JWT-based authentication with farmer and expert role management
- Production PWA with complete service worker and push notification support
- Real-time payment gateway integration for marketplace transactions
- WhatsApp API integration for instant alert delivery to farmers
- LLM-based advisory using large language models for richer chatbot responses
- Integration with national crop insurance and government scheme databases
- Advanced pest detection module using object detection models
- Weather API integration for location-specific crop advisory
- Multi-modal interaction: voice input in addition to voice output

Methodology

6.1 System Architecture Overview

The Krishi system follows a modular, layered architecture designed for offline-first AI-powered agricultural assistance. The system integrates image capture, disease detection, multilingual advisory, marketplace, expert consultation, community forum, and an AI chatbot into a cohesive unified platform.

High-Level System Workflow

The workflow proceeds as follows:

- Farmer opens the Krishi PWA app on a mobile browser
- Leaf image is captured via camera or uploaded from device gallery
- Image is preprocessed (resized to 224×224, normalized) locally in the browser
- TensorFlow.js model performs inference locally without internet (offline-first)
- Disease prediction with confidence score is displayed in the selected language
- Chemical and organic treatment recommendations are retrieved from local knowledge base
- Voice synthesis outputs results in the farmer's preferred language
- Farmer can consult the AI chatbot, book an expert, post to the community, or access the marketplace
- All online interactions sync with MongoDB Atlas when connectivity is available
- Historical data is stored for analytics and repeat use

6.2 Detailed Layered Methodology

6.2.1 Layer 1: Data Collection and Image Input

Functionality:

- Accept image input via camera capture or file upload from the device gallery
- Accept optional manual inputs such as crop type, symptoms, and region
- Provide a simplified mobile-first UI requiring minimal steps for image submission
- Support offline data capture with automatic sync on connectivity restoration
- Ensure real-time field data accessibility for non-technical users

Technologies: React 18, Web Camera API, HTML5 File Input

6.2.2 Layer 2: Image Preprocessing

Functionality:

- Resize uploaded images to 224×224 pixels for model compatibility
- Normalize pixel values to the [0, 1] range for consistent inference
- Apply quality checks for image clarity and sufficient leaf visibility
- Handle multi-resolution inputs with adaptive preprocessing

Technologies: TensorFlow.js tensor operations, HTML5 Canvas API

6.2.3 Layer 3: AI Disease Detection

Functionality:

- Load TensorFlow.js model (model.json + .bin files) cached in browser on first use
- Run inference locally in the browser with no server round-trip required
- Classify the leaf image across 15+ disease categories
- Output disease prediction with confidence score for farmer transparency
- Fall back to rule-based pattern matching from plantDiseases.js when model is unavailable

Key Parameters:

- Confidence threshold: 0.6 (adjustable per crop type)
- Input resolution: 224×224
- Output: Disease label + confidence percentage

Technologies: TensorFlow.js, tfjs-converter, React 18

6.2.4 Layer 4: AI Recommendation Engine

Functionality:

- Retrieve disease description, causes, and risk factors from plantDiseases.js knowledge base
- Provide chemical treatment recommendations (fertilizers, fungicides, pesticides)
- Suggest organic and natural remedies as alternative treatment options
- Generate prevention tips tailored to the detected disease
- Future scope: LLM-based multi-language advisory for richer contextual explanations

Technologies: Node.js knowledge base (plantDiseases.js, agriKnowledge.js), React state management

6.2.5 Layer 5: Multilingual Output and Voice Synthesis

Functionality:

- Render all UI text in the selected language: disease results, recommendations, forum, marketplace
- Support English, Hindi, and Kannada across all six core modules
- Generate voice synthesis output using the Web Speech API with language-specific voice selection
- Enable language switching at runtime without data loss or page reload

Technologies: Web Speech API, i18n translation layer, React context for language state

6.2.6 Layer 6: Offline-First PWA Architecture

Functionality:

- Implement Progressive Web App using service workers for asset and API caching
- Apply cache-first strategy for fast loading and offline access
- Store plantDiseases.js and agriKnowledge.js locally for offline disease lookup and chatbot
- Enable app installation on home screen like a native mobile application
- Achieve Lighthouse performance score of 98/100

Technologies: Service Workers, Cache API, Workbox, Vite PWA Plugin

6.2.7 Layer 7: AI Chatbot

Functionality:

- Provide 24/7 agricultural advisory through a conversational AI chatbot
- Query agriKnowledge.js for offline response when internet is unavailable
- Support multilingual query input and voice synthesis response output
- Handle agricultural questions on crop care, soil, fertilizers, pests, and weather

Technologies: Node.js /api/chat REST endpoint, agriKnowledge.js, React chat UI

6.2.8 Layer 8: Expert Consultation Module

Functionality:

- Display directory of certified agricultural experts with profiles and specializations
- Enable farmers to book consultation sessions with preferred experts
- Support consultation request management and status tracking
- Store expert data in MongoDB Atlas experts collection

Technologies: Node.js /api/consult REST endpoint, MongoDB Atlas, React

6.2.9 Layer 9: Peer-to-Peer Marketplace

Functionality:

- Enable farmers to list agricultural produce and inputs for sale
- Provide product search, filter, and inquiry management features
- Display product listings with images, prices, and seller contact information
- Store marketplace data in MongoDB Atlas products collection

Technologies: Node.js /api/market REST endpoint, MongoDB Atlas, Multer for image uploads

6.2.10 Layer 10: Community Forum

Functionality:

- Enable farmers to create posts, share knowledge, and ask questions
- Support threaded replies and community discussion
- Moderate content through reporting and admin review features
- Store forum data in MongoDB Atlas posts collection

Technologies: Node.js /api/community REST endpoint, MongoDB Atlas, React

6.2.11 Layer 11: Data Storage and Retrieval

Functionality:

- Store all transactional data in MongoDB Atlas cloud database
- Cache frequently accessed knowledge base data for fast offline access
- Provide efficient REST API querying for marketplace, community, and consultation modules
- Implement data validation and sanitization at the API layer

Databases:

- MongoDB Atlas: experts, products, and posts collections (primary database)

- Browser Cache: plantDiseases.js, agriKnowledge.js, TensorFlow.js model files
- LocalStorage: language preference, recent scan history (future enhancement)

6.3 System Integration Architecture

The system components communicate via:

- REST APIs: Synchronous communication between React frontend and Node.js/Express backend
- TensorFlow.js: In-browser model inference with no server round-trip for disease detection
- Service Workers: Offline caching and background sync between PWA and MongoDB Atlas
- MongoDB Atlas: Cloud-hosted persistent storage for all transactional data
- Web Speech API: Browser-native voice synthesis for multilingual audio output

6.4 Processing Pipeline

End-to-End Processing Flow:

1. Input: Camera or file upload → Image capture
2. Preprocessing: Image → Resize 224×224 → Normalize pixels
3. Detection: Preprocessed tensor → TensorFlow.js model → Disease label + confidence
4. Advisory: Disease label → plantDiseases.js lookup → Treatment recommendations
5. Output: Results → Multilingual rendering → Web Speech API voice output
6. Chatbot: User query → agriKnowledge.js → /api/chat → Response + voice
7. Marketplace: Product listing → /api/market → MongoDB Atlas → Display
8. Consultation: Booking request → /api/consult → Expert notification
9. Community: Post creation → /api/community → Forum display
10. Analytics: Usage data → MongoDB Atlas → Future admin dashboard

6.5 Technologies and Tools

6.5.1 Frontend

- React 18 with functional components and hooks
- Tailwind CSS with glassmorphism design system
- Web Speech API for voice synthesis
- TensorFlow.js for in-browser AI inference
- Service Workers and Cache API for PWA offline support

6.5.2 Backend Development

- Node.js with Express.js for REST API
- Mongoose ODM for MongoDB data modeling
- Multer for multipart image upload handling
- CORS, dotenv for configuration management

6.5.3 Database and Storage

- MongoDB Atlas (cloud-hosted NoSQL database)
- Browser Cache API (offline knowledge base and model caching)

6.5.4 DevOps and Deployment

- Vercel (frontend and serverless API deployment)
- GitHub for version control and CI/CD pipeline

Expected Outcomes

The proposed Krishi system is expected to deliver a comprehensive solution for AI-powered crop disease detection and integrated agricultural assistance. The key expected outcomes are:

7.1 Functional Outcomes

1. Real-Time Offline Disease Detection

- Automatically classify crop diseases from leaf images without internet connection
- Achieve approximately 92% accuracy across 15+ disease categories
- Complete scan-to-result flow in under 3 seconds

2. Actionable Treatment Recommendations

- Provide chemical and organic treatment options for every detected disease
- Include prevention tips and risk factor explanations
- Deliver all recommendations in the farmer's selected language with voice output

3. Multilingual Accessibility

- Full UI support for English, Hindi, and Kannada
- Voice synthesis in all three languages via Web Speech API
- Runtime language switching without data loss

4. Integrated Farmer Platform

- Six fully functional core modules: Scan, Chatbot, Market, Consult, Community, Multilingual UI
- Seamless navigation across modules within a single application
- Consistent glassmorphism UI design across all modules

5. Offline-First PWA

- Full functionality for disease diagnosis in zero-connectivity environments
- Lighthouse performance score of 98/100
- Installable as a native-like app on mobile devices

6. Scalable REST API Backend

- Four production REST API endpoints: /api/chat, /api/community, /api/market, /api/consult
- MongoDB Atlas database with experts, products, and posts collections
- Support for multiple concurrent users with efficient API handling

7.2 Non-Functional Outcomes

1. Performance

- End-to-end scan completion: less than 3 seconds
- API response time: less than 500ms per request
- App initial load time: less than 2 seconds on 3G connection

2. Reliability

- Offline disease detection available 100% of the time regardless of connectivity
- Graceful degradation from AI detection to rule-based fallback when model is unavailable
- Automatic recovery of marketplace and community features on connectivity restoration

3. Security

- HTTPS-only deployment via Vercel
- Input sanitization at all API endpoints
- Future: JWT-based authentication for user accounts and role management

4. Usability

- Minimal-step interface requiring no technical training for core disease scan workflow
- Voice output for non-literate users
- Mobile-responsive design optimized for low-end Android smartphones

7.3 Deliverables

7.3.1 Software Deliverables

- Functional React 18 + Node.js + MongoDB web application with 6 core modules
- TensorFlow.js-based in-browser crop disease detection module
- Offline knowledge base: plantDiseases.js (15+ diseases), agriKnowledge.js
- Four REST API endpoints: /api/chat, /api/community, /api/market, /api/consult
- MongoDB Atlas schema and initialization scripts for experts, products, posts
- PWA service worker configuration for offline caching
- Multilingual UI with Web Speech API voice synthesis in English, Hindi, Kannada
- Vercel deployment configuration for production hosting

7.3.2 Documentation Deliverables

- Project synopsis and technical specification
- System design document with architecture diagrams and data flow
- API documentation for all REST endpoints
- Installation and deployment guide for Vercel and MongoDB Atlas
- User manual for all six core modules
- Literature survey with 30 references
- Performance analysis and Lighthouse audit report
- Final project report
- Project demonstration video
- Presentation slides

Work Plan / Timeline

Phase	Task	Duration	Key Milestones
Phase 1	Literature Review & Analysis	Week 1-2	Study existing systems, architectures and SOTA models Understanding requirements, challenges
Phase 2	Problem Definition & Design	Week 3-4	Define objectives, system requirements Architecture and design, technology selection
Phase 3	Implementation	Week 5-10	Video input module development Detection and tracking implementation Re-identification and matching module Alert system and dashboard
Phase 4	Testing & Evaluation	Week 11-12	Unit and integration testing Performance analysis and optimization
Phase 5	Documentation & Presentation	Week 13-14	Final documentation and report writing Presentation preparation and demo

8.1 Detailed Timeline

8.1.1 Phase 1: Literature Review (Week 1-2)

- Review papers on plant disease detection, edge AI, multilingual NLP, and PWA architecture
- Study TensorFlow.js, MobileNet, and PlantVillage dataset implementations
- Analyze existing agricultural apps and their limitations
- Identify suitable datasets and evaluation metrics for disease detection accuracy

8.1.2 Phase 2: Design & Planning (Week 3-4)

- Finalize system architecture and module specifications for all six core modules
- Select technology stack: React 18, Node.js, MongoDB Atlas, TensorFlow.js, Vercel
- Design MongoDB schema for experts, products, and posts collections
- Design REST API specification for all four endpoints
- Create data flow diagrams and UI wireframes

8.1.3 Phase 3: Implementation (Week 5-10)

- Develop React 18 frontend with Tailwind CSS glassmorphism UI
- Integrate TensorFlow.js model for in-browser crop disease detection
- Build plantDiseases.js offline knowledge base with 15+ disease categories
- Implement Web Speech API multilingual voice synthesis module
- Set up PWA service worker and cache-first offline strategy
- Develop Node.js/Express REST API with all four endpoints

- Configure MongoDB Atlas and implement Mongoose schemas
- Build chatbot module with agriKnowledge.js knowledge base
- Develop marketplace, consultation, and community forum modules
- Deploy to Vercel for staging and testing

8.1.4 Phase 4: Testing & Optimization (Week 11-12)

- Perform unit testing of disease detection, API endpoints, and UI components
- Conduct integration testing of complete scan-to-recommendation pipeline
- Run Lighthouse audit and optimize for 98/100 performance score
- Test offline functionality in simulated low-connectivity environments
- Stress test API endpoints with concurrent user simulation
- Fix bugs and edge cases identified during testing

8.1.5 Phase 5: Documentation & Presentation (Week 13-14)

- Write comprehensive technical documentation and project synopsis
- Create API documentation for all REST endpoints
- Prepare deployment guide for Vercel and MongoDB Atlas
- Write final project report with all chapters and literature survey
- Create presentation slides and prepare live demonstration
- Record video demonstration of all six core modules

References

1. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). "Using Deep Learning for Image-Based Plant Disease Detection." Frontiers in Plant Science, 7, 1419.
2. Too, E. C., Yujian, L., Njuki, S., & Yingchun, L. (2022). "A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification." Computers and Electronics in Agriculture, 161, 272–279.
3. Ferentinos, K. P. (2018). "Deep Learning Models for Plant Disease Detection and Diagnosis." Computers and Electronics in Agriculture, 145, 311–318.
4. Hughes, D. P., & Salathé, M. (2015). "An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics." arXiv preprint arXiv:1511.08060.
5. Brahimi, M., Boukhalfa, K., & Moussaoui, A. (2017). "Deep Learning for Tomato Diseases: Classification and Symptoms Visualization." Applied Artificial Intelligence, 31(4), 299–315.
6. Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). "Deep Learning in Agriculture: A Survey." Computers and Electronics in Agriculture, 147, 70–90.
7. Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2020). "PlantDoc: A Dataset for Visual Plant Disease Detection." ACM India Joint International Conference on Data Science and Management of Data.
8. Durmuş, H., Güneş, E. O., & Kırcı, M. (2017). "Disease Detection on the Leaves of the Tomato Plants by Using Deep Learning." 6th International Conference on Agro-Geoinformatics.
9. Tan, M., & Le, Q. V. (2019). "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." International Conference on Machine Learning (ICML).
10. Zhang, C., et al. (2020). "Mobile-Based Plant Disease Detection Using Deep Learning." IEEE Access.
11. Sharma, A., et al. (2024). "Edge AI for Offline Plant Disease Detection in Rural Agricultural Environments." IEEE Transactions on AgriInformatics.
12. Kumar, R., et al. (2023). "AI Chatbot for Farmer Assistance: An NLP-Based Approach for Agricultural Advisory Systems." International Journal of Agricultural Technology.
13. Patel, V., et al. (2023). "AI-Based Smart Farming System Using IoT and Deep Learning." Journal of King Saud University – Computer and Information Sciences.
14. Reddy, K., et al. (2024). "Multi-Language AI Systems for Agriculture in India." Preprint.
15. Lee, C., et al. (2024). "Edge Computing in Smart Farming for Offline AI Processing." IEEE Internet of Things Journal.
16. Google Developers. (2018). "TensorFlow.js: Machine Learning for the Web and Beyond." Technical Paper, Google Brain.
17. (2020). "Progressive Web Apps for Offline Web Systems." Technical Paper.
18. Gupta, S., et al. (2024). "Offline AI for Agriculture Using Edge Computing." Preprint.
19. Mishra, P., et al. (2023). "Deep Learning-Based Smart Farming Systems: A Comprehensive Review." Preprint.
20. Li, X., et al. (2022). "CNN-Based Crop Health Monitoring System Using Leaf Image Analysis." Preprint.
21. Barbedo, J. G. A. (2018). "Factors Influencing the Use of Deep Learning for Plant Disease Recognition." Biosystems Engineering, 172, 84–91.

22. Rumpf, T., et al. (2010). "Early Detection and Classification of Plant Diseases with Support Vector Machines." Computers and Electronics in Agriculture, 74(1), 91–99.
23. Islam, M., et al. (2021). "Hybrid CNN-SVM Approach for Plant Disease Detection." Agriculture, 11(7), 641.
24. Rajalakshmi, P., et al. (2021). "IoT-Based Crop Monitoring System with Real-Time Alerts." International Conference on Smart Technologies.
25. Verma, R., et al. (2023). "Real-Time Plant Monitoring System Using IoT and Cloud Computing." Sustainable Computing: Informatics and Systems.
26. Kaur, G., et al. (2021). "Smart Irrigation System Using IoT for Water Usage Optimization." International Journal of Computer Applications.
27. Chen, Y., et al. (2023). "AI-Based Pest Detection System Using CNN with Early Warning Capabilities." Precision Agriculture.
28. Zhang, Y., et al. (2024). "Lightweight CNN for Mobile Deployment in Agricultural Applications." IEEE Transactions on Mobile Computing.
29. Rao, S., et al. (2022). "Agricultural Decision Support System Using AI and Data Analytics." Decision Support Systems.
30. FAO Researchers. (2023). "AI in Sustainable Agriculture: Global Impact and Implementation Challenges." Food and Agriculture Organization of the United Nations.